

E-Commerce Platform - Architecture Runbook

Document Version: 1.0

Last Updated: January 2025

Classification: Production Documentation

Target Audience: DevOps Engineers, Solution Architects, Security Teams

Table of Contents

1. [Executive Summary](#)
 2. [System Overview](#)
 3. [Architecture Components](#)
 4. [API Endpoints](#)
 5. [Data Architecture](#)
 6. [Security Architecture](#)
 7. [Deployment Guide](#)
 8. [Operational Procedures](#)
 9. [Monitoring and Logging](#)
 10. [Troubleshooting](#)
-

Executive Summary

This document describes the architecture and operational procedures for our enterprise e-commerce platform. The system is built on AWS serverless technologies, providing a scalable and cost-effective solution for managing users, products, orders, and payment processing.

Key Metrics

- **11 API Endpoints** across 4 business domains
- **11 Lambda Functions** handling business logic
- **4 DynamoDB Tables** for data persistence
- **Serverless Architecture** for automatic scaling
- **REST API** for client integration

Technology Stack

- **Compute:** AWS Lambda (Python 3.11)
- **API Management:** Amazon API Gateway
- **Data Storage:** Amazon DynamoDB

- **Monitoring:** Amazon CloudWatch
 - **Infrastructure:** Terraform (Infrastructure as Code)
-

System Overview

Business Context

The e-commerce platform provides comprehensive functionality for online retail operations:

- **Customer Management:** User registration, profile management, and account operations
- **Product Catalog:** Product listing, search, and detail retrieval
- **Order Processing:** Order creation, tracking, and history management
- **Payment Processing:** Payment transaction handling and financial reporting

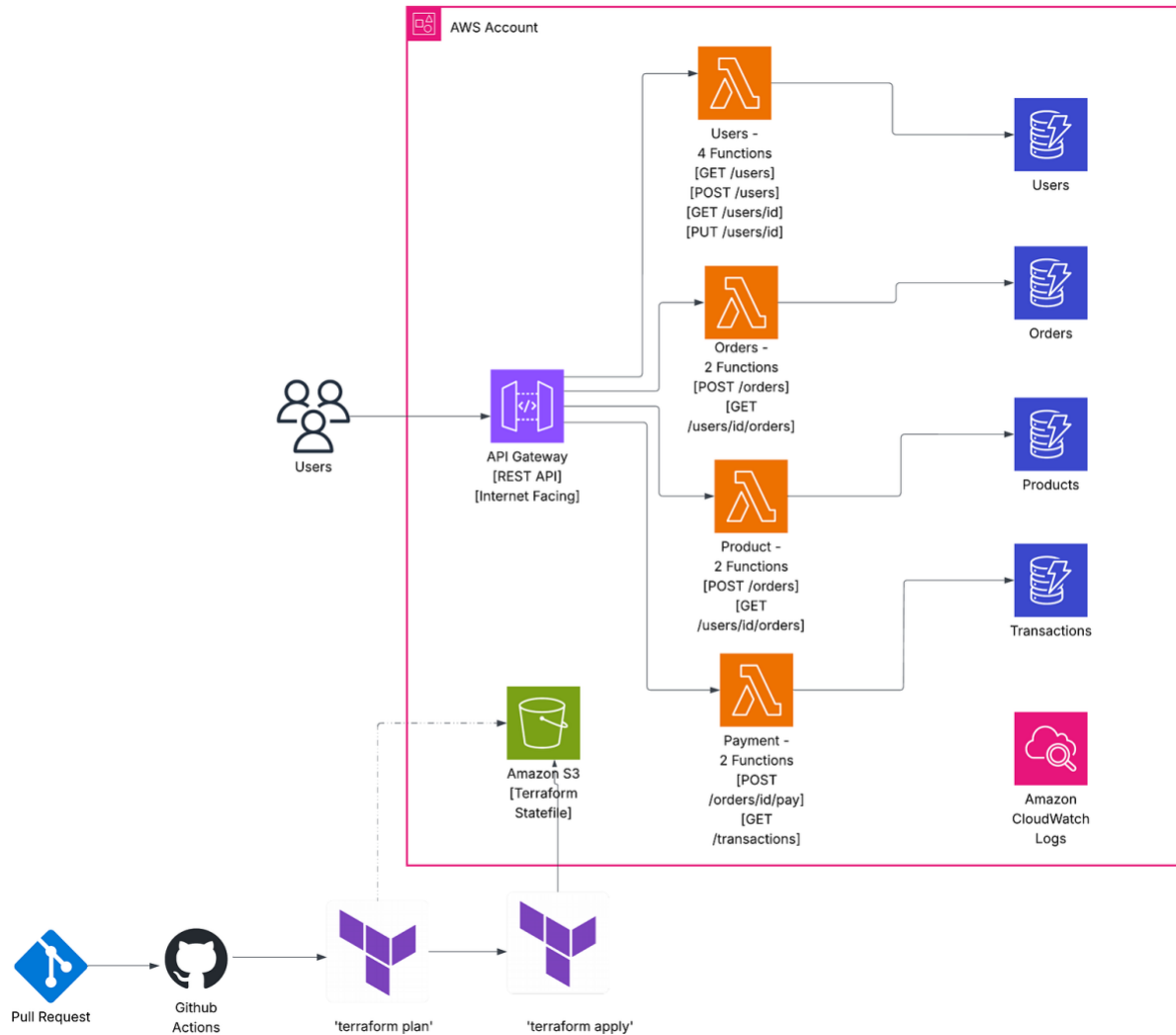
Architecture Principles

The platform is designed with the following principles:

- **Serverless-First:** Leveraging AWS Lambda for compute to minimize operational overhead
 - **API-Driven:** RESTful API design for easy integration with web and mobile clients
 - **Scalable:** Auto-scaling capabilities through serverless architecture
 - **Cost-Optimized:** Pay-per-use pricing model with DynamoDB and Lambda
 - **Cloud-Native:** Built specifically for AWS cloud infrastructure
-

Architecture Components

Architecture Diagram



High-Level Architecture

Internet Clients



API Gateway (REST API)



Lambda Functions (11 functions)



DynamoDB Tables (4 tables)



CloudWatch Logs

Component Details

API Gateway

- **Type:** REST API
- **Endpoints:** 11 total endpoints
- **Protocol:** HTTPS
- **Stage:** Production deployment stage
- **CORS:** Enabled for cross-origin requests

Lambda Functions

- **Runtime:** Python 3.11
- **Deployment:** Single deployment package for all functions
- **Execution Role:** Shared IAM role across functions
- **Timeout:** Varies by function (15s - 180s)
- **Memory:** Varies by function (128MB - 3008MB)

DynamoDB Tables

- **Billing:** Mixed (Provisioned and Pay-per-request)
- **Capacity:** Configured per table based on expected load
- **Indexes:** Global Secondary Indexes for query optimization
- **Backup:** Standard AWS backup capabilities

CloudWatch

- **Logs:** Centralized logging for all Lambda functions
 - **Retention:** Configured per function log group
 - **Metrics:** Standard Lambda and DynamoDB metrics
-

API Endpoints

User Management Endpoints

List Users

- **Method:** GET
- **Path:** /users
- **Function:** list-users-function
- **Description:** Retrieves a list of all users with pagination support
- **Query Parameters:**
 - **limit:** Maximum number of results (default: 1000)
 - **lastKey:** Pagination token for next page

Get User

- **Method:** GET

- **Path:** /users/{userId}
- **Function:** get-user-function
- **Description:** Retrieves detailed information for a specific user
- **Path Parameters:**
 - userId: Unique user identifier

Create User

- **Method:** POST
- **Path:** /users
- **Function:** create-user-function
- **Description:** Creates a new user account
- **Request Body:**

```
{
  "name": "string",
  "email": "string",
  "status": "string"
}
```

Update User

- **Method:** PUT
- **Path:** /users/{userId}
- **Function:** update-user-function
- **Description:** Updates an existing user's information
- **Path Parameters:**
 - userId: Unique user identifier
- **Request Body:** Partial user object with fields to update

Delete User

- **Method:** DELETE
- **Path:** /users/{userId}
- **Function:** delete-user-function
- **Description:** Removes a user from the system
- **Path Parameters:**
 - userId: Unique user identifier

Product Catalog Endpoints

List Products

- **Method:** GET
- **Path:** /products

- **Function:** list-products-function
- **Description:** Retrieves product catalog with optional filtering
- **Query Parameters:**
 - category: Filter by product category
 - limit: Maximum number of results (default: 500)

Get Product

- **Method:** GET
- **Path:** /products/{productId}
- **Function:** get-product-function
- **Description:** Retrieves detailed information for a specific product
- **Path Parameters:**
 - productId: Unique product identifier

Order Management Endpoints

Create Order

- **Method:** POST
- **Path:** /orders
- **Function:** create-order-function
- **Description:** Creates a new order for a user
- **Request Body:**

```
{
  "userId": "string",
  "items": [
    {
      "productId": "string",
      "quantity": number
    }
  ]
}
```

Get User Orders

- **Method:** GET
- **Path:** /users/{userId}/orders
- **Function:** get-user-orders-function
- **Description:** Retrieves order history for a specific user
- **Path Parameters:**
 - userId: Unique user identifier
- **Query Parameters:**

- status: Filter by order status

Payment Processing Endpoints

Process Payment

- **Method:** POST
- **Path:** /orders/{orderId}/payment
- **Function:** process-payment-function
- **Description:** Processes payment for an order
- **Path Parameters:**
 - orderId: Unique order identifier
- **Request Body:**

```
{  
  "userId": "string",  
  "amount": number,  
  "paymentMethod": "string",  
  "cardNumber": "string",  
  "cvv": "string",  
  "expiryDate": "string"  
}
```

Get Transactions

- **Method:** GET
- **Path:** /transactions
- **Function:** get-transactions-function
- **Description:** Retrieves financial transaction records
- **Query Parameters:**
 - userId: Filter by user
 - startDate: Filter by date range
 - endDate: Filter by date range
 - limit: Maximum number of results (default: 1000)

Data Architecture

DynamoDB Tables

Users Table

- **Name:** {environment}-users-table
- **Billing Mode:** Provisioned

- **Read Capacity:** 5 RCU
- **Write Capacity:** 5 WCU
- **Primary Key:** userId (String)
- **Attributes:** userId, name, email, status, createdAt
- **Use Case:** Stores user account information and profiles

Products Table

- **Name:** {environment}-products-table
- **Billing Mode:** Provisioned
- **Read Capacity:** 10 RCU
- **Write Capacity:** 2 WCU
- **Primary Key:** productId (String)
- **Global Secondary Index:** CategoryIndex
 - **Hash Key:** category
 - **Read Capacity:** 3 RCU
 - **Write Capacity:** 1 WCU
- **Attributes:** productId, name, category, price, stock, description
- **Use Case:** Product catalog and inventory management

Orders Table

- **Name:** {environment}-orders-table
- **Billing Mode:** Pay-per-request
- **Primary Key:**
 - **Hash Key:** orderId (String)
 - **Range Key:** userId (String)
- **Global Secondary Index:** UserOrdersIndex
 - **Hash Key:** userId
 - **Range Key:** orderStatus
- **Attributes:** orderId, userId, items, totalAmount, orderStatus, createdAt, updatedAt
- **Use Case:** Order tracking and management

Transactions Table

- **Name:** {environment}-transactions-table
- **Billing Mode:** Provisioned
- **Read Capacity:** 100 RCU
- **Write Capacity:** 50 WCU
- **Primary Key:**
 - **Hash Key:** transactionId (String)
 - **Range Key:** timestamp (String)
- **Global Secondary Index:** UserTransactionsIndex

- **Hash Key:** userId
- **Range Key:** timestamp
- **Read Capacity:** 50 RCU
- **Write Capacity:** 25 WCU
- **Attributes:** transactionId, orderId, userId, amount, paymentMethod, status, timestamp
- **Use Case:** Financial transaction records and reporting

Data Flow Patterns

User Registration Flow

1. Client sends POST request to /users
2. API Gateway routes to create-user-function
3. Lambda generates unique userId
4. User data written to Users table
5. Response returned to client

Order Creation Flow

1. Client sends POST request to /orders
2. API Gateway routes to create-order-function
3. Lambda validates user exists in Users table
4. Lambda retrieves product details from Products table
5. Lambda calculates order total
6. Order record written to Orders table
7. Response returned to client

Payment Processing Flow

1. Client sends POST request to /orders/{orderId}/payment
2. API Gateway routes to process-payment-function
3. Lambda retrieves order from Orders table
4. Lambda processes payment information
5. Transaction record written to Transactions table
6. Order status updated in Orders table
7. Response returned to client

Security Architecture

IAM Configuration

Lambda Execution Role

- **Role Name:** {environment}-user-api-lambda-role
- **Trusted Entity:** lambda.amazonaws.com

- **Attached Policies:**
 - AWSLambdaBasicExecutionRole (AWS Managed)
 - Custom DynamoDB access policy
 - Custom CloudWatch logging policy

DynamoDB Access Policy

The Lambda functions have the following DynamoDB permissions: - dynamodb:GetItem - dynamodb:PutItem - dynamodb:UpdateItem - dynamodb>DeleteItem - dynamodb:Query - dynamodb:Scan

Resources: All four DynamoDB tables and their indexes

CloudWatch Logging Policy

Lambda functions can write logs to CloudWatch: - logs:CreateLogGroup - logs:CreateLogStream - logs:PutLogEvents

API Gateway Configuration

CORS Settings

- **Allowed Origins:** *
- **Allowed Methods:** GET, POST, PUT, DELETE, OPTIONS
- **Allowed Headers:** Content-Type, Authorization

Request/Response Configuration

- **Content Type:** application/json
- **Integration Type:** AWS_PROXY (Lambda Proxy Integration)
- **Timeout:** 29 seconds (API Gateway maximum)

Operational Procedures

Monitoring

CloudWatch Metrics

Monitor the following key metrics:

Lambda Functions: - Invocations - Duration - Errors - Throttles - Concurrent Executions

DynamoDB Tables: - ConsumedReadCapacityUnits - ConsumedWriteCapacityUnits - UserErrors - SystemErrors - ThrottledRequests

API Gateway: - Count (total requests) - 4XXError - 5XXError - Latency

Log Analysis

Lambda function logs are available in CloudWatch Logs:

View logs for a specific function

```
aws logs tail /aws/lambda/prod-get-user-function --follow
```

Search for errors

```
aws logs filter-log-events \  
  --log-group-name /aws/lambda/prod-process-payment-function \  
  --filter-pattern "ERROR"
```

Backup and Recovery

DynamoDB Backups

DynamoDB tables use AWS's built-in backup capabilities:

Create on-demand backup

```
aws dynamodb create-backup \  
  --table-name prod-users-table \  
  --backup-name users-backup-$(date +%Y%m%d)
```

Disaster Recovery

In case of data loss or corruption:

1. Identify the affected table
2. Restore from the most recent backup
3. Verify data integrity
4. Update application if necessary

Scaling Considerations

Lambda Scaling

- Lambda automatically scales based on incoming requests
- No manual intervention required for compute scaling

DynamoDB Scaling

- **Provisioned Tables:** Monitor capacity metrics and adjust RCU/WCU as needed
- **Pay-per-request Tables:** Automatically scales with demand

Resource Naming Convention

All resources follow this naming pattern:

{environment}-{resource-type}-{domain}

Examples: - prod-users-table - prod-get-user-function - /aws/lambda/prod-create-order-function

Terraform State Management

Terraform state is stored in S3: - **Bucket:** terraform-statefile-bucket-0503 - **Key:** security-agent-demo/terraform.tfstate - **Region:** us-east-1

Monitoring and Logging

CloudWatch Log Groups

Each Lambda function has a dedicated log group:

Function	Log Group	Retention
get-user	/aws/lambda/{env}-get-user-function	60 days
create-user	/aws/lambda/{env}-create-user-function	30 days
update-user	/aws/lambda/{env}-update-user-function	14 days
delete-user	/aws/lambda/{env}-delete-user-function	7 days
list-users	/aws/lambda/{env}-list-users-function	90 days
list-products	/aws/lambda/{env}-list-products-function	30 days
get-product	/aws/lambda/{env}-get-product-function	14 days
create-order	/aws/lambda/{env}-create-order-function	365 days
get-user-orders	/aws/lambda/{env}-get-user-orders-function	60 days
process-payment	/aws/lambda/{env}-process-payment-function	2555 days
get-transactions	/aws/lambda/{env}-get-transactions-function	1 day

Log Format

Lambda functions log in JSON format for easy parsing:

```
{
  "timestamp": "2025-01-14T10:30:00Z",
  "level": "INFO",
  "requestId": "abc-123-def",
  "message": "Processing request for user lookup",
  "userId": "user123"
}
```

Performance Monitoring

Key performance indicators to monitor:

- **API Response Time:** Target < 500ms for GET requests
 - **Lambda Duration:** Monitor against configured timeout
 - **DynamoDB Latency:** Target < 10ms for GetItem operations
 - **Error Rate:** Target < 0.1% for all endpoints
-

Troubleshooting

Common Issues

DynamoDB Throttling

Symptoms: - ProvisionedThroughputExceededException errors in logs - Increased latency for database operations

Resolution: 1. Check CloudWatch metrics for consumed capacity 2. Increase provisioned capacity for affected table 3. Consider switching to pay-per-request billing mode

Lambda Timeout

Symptoms: - Task timed out after X seconds in logs - 504 Gateway Timeout responses from API

Resolution: 1. Review function logs to identify slow operations 2. Optimize database queries 3. Increase function timeout if necessary 4. Consider breaking complex operations into smaller functions

API Gateway Errors

Symptoms: - 4XX or 5XX errors from API endpoints - CORS-related errors in browser console

Resolution: 1. Verify request format matches API specification 2. Check Lambda function logs for detailed error messages 3. Verify CORS configuration if browser-based 4. Ensure API Gateway has permission to invoke Lambda

Health Checks

Perform regular health checks:

Test all endpoints

```
curl $API_ENDPOINT/users
```

```
curl $API_ENDPOINT/products
```

```
curl $API_ENDPOINT/transactions
```

Check Lambda function status

```
aws lambda get-function --function-name prod-get-user-function
```

Check DynamoDB table status

```
aws dynamodb describe-table --table-name prod-users-table
```

Support Escalation

For issues that cannot be resolved:

1. Gather relevant logs and metrics
2. Document steps to reproduce
3. Contact AWS Support with case details
4. Escalate to development team if application-specific

Document Control

This architecture runbook provides comprehensive documentation for the e-commerce platform infrastructure and operations. Keep this document updated as the system evolves.